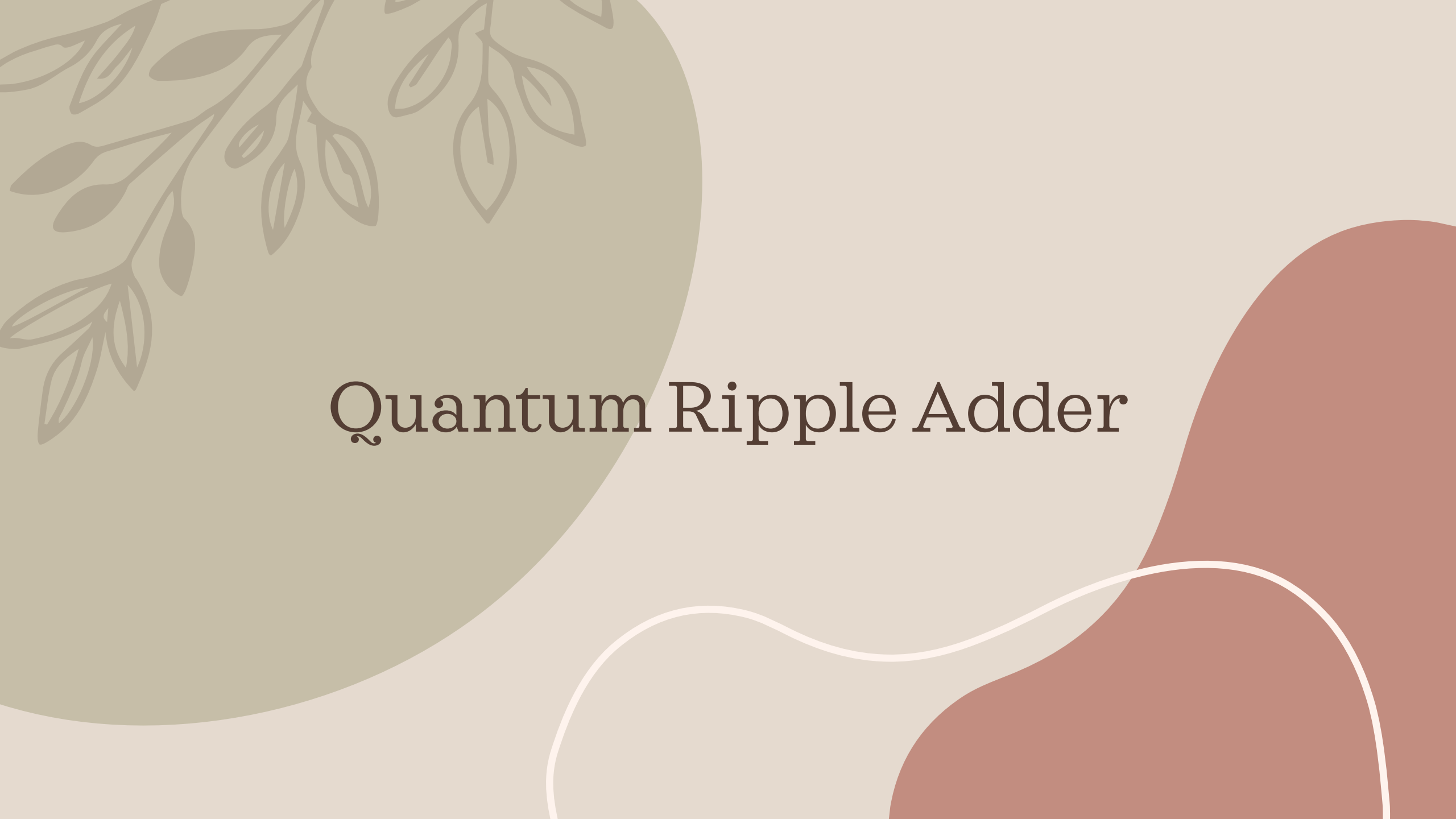




Quantum Ripple Adder

What is Ripple Adder

- A ripple carry adder is a digital circuit that adds two binary numbers by cascading full adders. The "ripple" refers to how the carry bit propagates from the least significant bit (LSB) to the most significant bit (MSB), like a ripple effect.

Key Components:

- **Full Adder:** Adds three bits (A, B, and carry-in) to produce a sum and carry-out
- **Half Adder:** Adds two bits (used only for the LSB position when there's no initial carry)
- **Carry Chain:** The carry-out from each stage becomes the carry-in for the next stage

Gate Definitions and Roles

Gate Definitions and Roles

Quantum Gates Used:

1. CNOT (Controlled-X) Gate:

1. Flips the target qubit if the control qubit is $|1\rangle$
2. Used for XOR operations and entanglement
3. Essential for implementing classical XOR logic

2. Toffoli (CCX) Gate:

1. Three-qubit gate that flips the target if both control qubits are $|1\rangle$
2. Implements classical AND logic
3. Used to generate carry bits

3. X Gate:

1. Bit flip gate (NOT operation)
2. Used for input preparation and corrections

How it Works?

How the Ripple Carry Adder Works

- The ripple carry adder operates by implementing the classical binary addition algorithm:

Bit-by-bit Addition: Starting from the least significant bit (LSB), each position adds:

- Bit from first number (A)
- Bit from second number (B)
- Carry from previous position

Full Adder Logic: For each bit position:

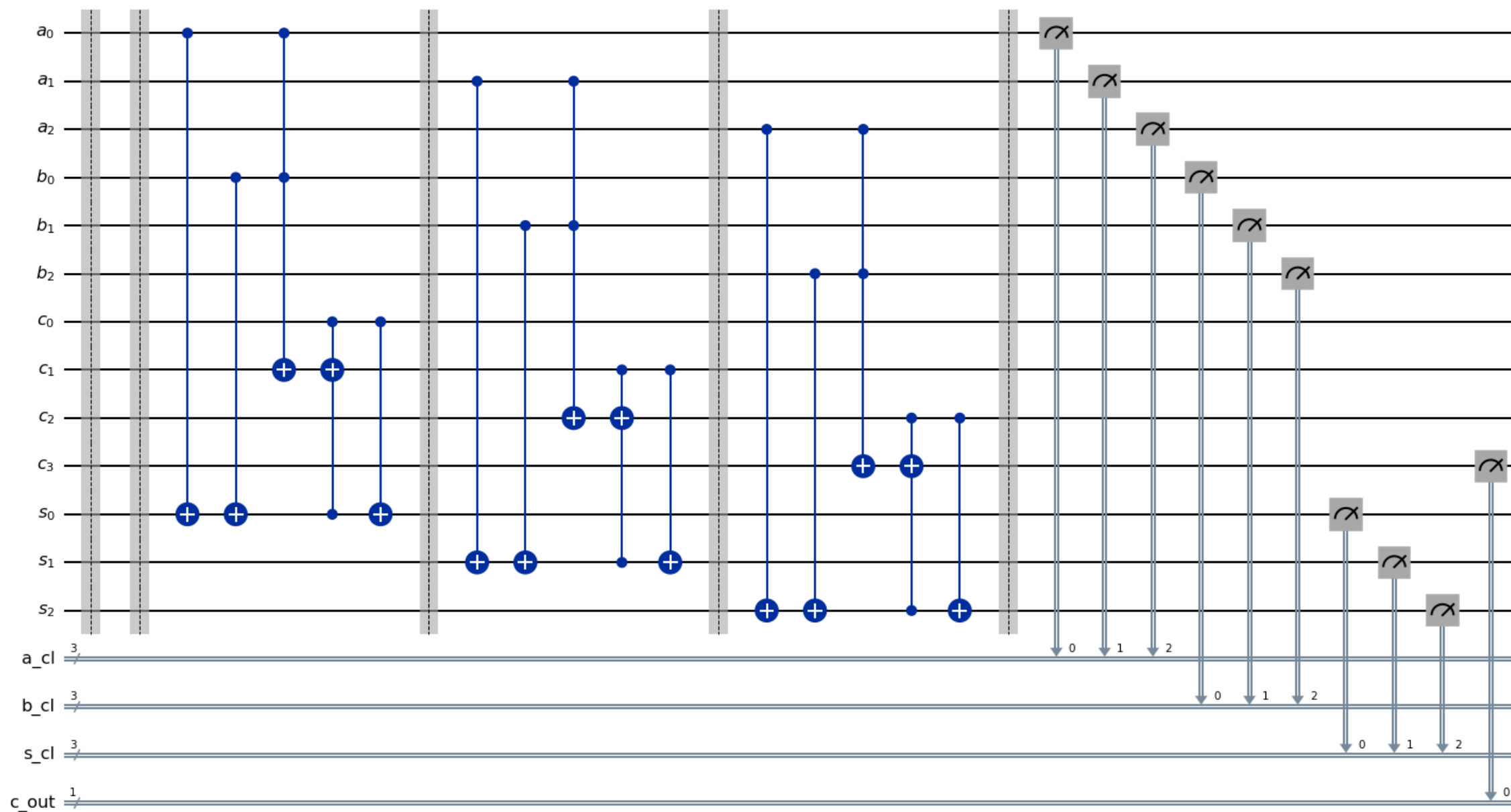
- **Sum bit:** $S = A \oplus B \oplus \text{Carry_in}$
- **Carry out:** $\text{Carry_out} = (A \wedge B) \vee (\text{Carry_in} \wedge (A \oplus B))$
- **Carry Propagation:** The carry-out from position i becomes the carry-in for position $i+1$

Quantum Implementation Details

Qubit Allocation:

- $a[0 : n-1]$: First input number qubits
- $b[0 : n-1]$: Second input number qubits
- $c[0 : n]$: Carry qubits (including final carry-out)
- $s[0 : n-1]$: Sum result qubits
- **Gate Operations:**
 - **CX gates** create the XOR operations needed for sum calculation
 - **CCX (Toffoli) gates** generate carry bits using AND logic
 - **Barrier instructions** organize the circuit for clarity

Circuit Diagram



Init

```
def __init__(self, n_bits=3):
```

```
    """
```

Initialize the quantum ripple adder

Args:

 n_bits (int): Number of bits for each input number

```
    """
```

```
    self.n_bits = n_bits
```

```
    self.circuit = None
```

```
    self.create_adder_circuit()
```

Create Adder Circuit

Create the quantum ripple carry adder circuit

Qubit allocation:

- $a[i]$: First input number (bits 0 to $n-1$)
- $b[i]$: Second input number (bits 0 to $n-1$)
- $c[i]$: Carry bits (bits 0 to n , where $c[0]$ is carry-in, $c[n]$ is final carry-out)
- $s[i]$: Sum output bits (bits 0 to $n-1$)

Create Circuit

```
def create_adder_circuit(self):  
    # Create quantum registers  
    a = QuantumRegister(self.n_bits, 'a') # First input  
    b = QuantumRegister(self.n_bits, 'b') # Second input  
    c = QuantumRegister(self.n_bits + 1, 'c') # Carry bits (including carry-out)  
    s = QuantumRegister(self.n_bits, 's') # Sum output  
    # Classical registers for measurement  
    a_cl = ClassicalRegister(self.n_bits, 'a_cl')  
    b_cl = ClassicalRegister(self.n_bits, 'b_cl')  
    s_cl = ClassicalRegister(self.n_bits, 's_cl')  
    c_out_cl = ClassicalRegister(1, 'c_out')  
    # Measure all qubits  
    self.circuit.measure(a, a_cl)  
    self.circuit.measure(b, b_cl)  
    self.circuit.measure(s, s_cl)  
    self.circuit.measure(c[self.n_bits], c_out_cl) # Final carry-out
```

Create Circuit ...

Create the circuit

```
self.circuit = QuantumCircuit(a, b, c, s, a_cl, b_cl, s_cl, c_out_cl)
```

Add a barrier for clarity

```
self.circuit.barrier()
```

```
self.circuit.label = "Input Preparation"
```

Implement the ripple carry adder logic

```
for i in range(self.n_bits):
```

```
    self.add_full_adder(i, a, b, c, s)
```

Add final barrier

```
self.circuit.barrier()
```

Full Adder

Add a full adder for the given bit position

Full Adder Logic:

$$\text{sum} = a \oplus b \oplus \text{carry_in}$$

$$\text{carry_out} = (a \wedge b) \vee (\text{carry_in} \wedge (a \oplus b))$$

Quantum Implementation:

1. Compute $a \oplus b$ and store in sum qubit
2. Compute carry_out using Toffoli gates
3. Apply final XOR with carry_in to get final

Full Adder

```
def add_full_adder(self, bit_position, a, b, c, s):  
    i = bit_position  
    # Label this section  
    self.circuit.barrier()  
    # Step 1: Compute  $a[i] \oplus b[i] \rightarrow s[i]$   
    self.circuit.cx(a[i], s[i])  
    self.circuit.cx(b[i], s[i])  
    # Step 2: Compute  $\text{carry\_out} = (a[i] \wedge b[i]) \vee (c[i] \wedge (a[i] \oplus b[i]))$   
    # First part:  $a[i] \wedge b[i] \rightarrow c[i+1]$   
    self.circuit.ccx(a[i], b[i], c[i+1])  
    # Second part:  $c[i] \wedge (a[i] \oplus b[i]) \rightarrow c[i+1]$   
    # We use s[i] which currently holds  $a[i] \oplus b[i]$   
    self.circuit.ccx(c[i], s[i], c[i+1])  
    # Step 3: Final sum  $= (a[i] \oplus b[i]) \oplus c[i] = s[i] \oplus c[i]$   
    self.circuit.cx(c[i], s[i])
```

Define Inputs

Set input values for the adder by applying X gates

Args:

a_val (int): First input number

b_val (int): Second input number

carry_in (int): Initial carry input (0 or 1)

Define Inputs

```
def set_input_values(self, a_val, b_val, carry_in=0):  
    # Reset circuit to clean state  
    self.create_adder_circuit()  
    # Convert to binary and set input qubits  
    a_binary = format(a_val, f'O{self.n_bits}b')[::-1] # Reverse for LSB first  
    b_binary = format(b_val, f'O{self.n_bits}b')[::-1]  
    # Create a new circuit with input preparation  
    input_prep = QuantumCircuit(self.circuit.qregs[0], self.circuit.qregs[1],  
                                self.circuit.qregs[2], self.circuit.qregs[3])
```

Define Inputs

Set input a

```
for i, bit in enumerate(a_binary):
```

```
    if bit == '1':
```

```
        input_prep.x(self.circuit.qregs[0][i]) # a register
```

Set input b

```
for i, bit in enumerate(b_binary):
```

```
    if bit == '1':
```

```
        input_prep.x(self.circuit.qregs[1][i]) # b register
```

Set carry-in

```
if carry_in == 1:
```

```
    input_prep.x(self.circuit.qregs[2][0]) # c register, first qubit
```

Combine input preparation with main circuit

```
full_circuit = input_prep.compose(self.circuit)
```

```
self.circuit = full_circuit
```

```
return self.circuit
```

Simulate

```
def simulate(self, shots=1024):  
    simulator = AerSimulator()  
    # Use the preset pass manager for optimization  
    pm = generate_preset_pass_manager(backend=simulator, optimization_level=1)  
    transpiled_circuit = pm.run(self.circuit)  
    # Run simulation  
    job = simulator.run(transpiled_circuit, shots=shots)  
    result = job.result()  
    counts = result.get_counts()  
    return counts
```

Demonstrate

Demonstration and testing

Demonstrate the quantum ripple carry adder with different examples

```
"""
```

```
print("=" * 60)
```

```
print("QUANTUM RIPPLE CARRY ADDER DEMONSTRATION")
```

```
print("=" * 60)
```

```
# Create a 3-bit adder
```

```
adder = QuantumRippleAdder(n_bits=3)
```

```
# Test different addition operations
```

```
test_cases = [
```

```
    (3, 2), # 011 + 010 = 101 (5)
```

```
    (5, 3), # 101 + 011 = 1000 (8, with carry)
```

```
    (7, 1), # 111 + 001 = 1000 (8, with carry)
```

```
    (0, 0), # 000 + 000 = 000 (0)
```

```
]
```

Demonstrate ...

```
for a_val, b_val in test_cases:
    print(f"\nTesting: {a_val} + {b_val}")
    print(f"Expected result: {a_val + b_val}")
    # Set up the circuit with input values
    adder.set_input_values(a_val, b_val)
    # Simulate
    counts = adder.simulate(shots=1024)
    # Analyze results
    adder.analyze_results(counts)
    print("-" * 40)
# Draw the circuit for the last test case
print("\nQuantum Circuit Diagram:")
print("(Showing circuit for the last test case: 7 + 1)")
```

Demonstrate ...

try:

```
# Try matplotlib output first
```

```
fig = adder.draw_circuit(output='mpl')
```

```
if fig:
```

```
    plt.figure(figsize=(15, 8))
```

```
    plt.tight_layout()
```

```
    print("Circuit diagram saved as matplotlib figure")
```

except:

```
# Fallback to text output
```

```
print("Matplotlib not available, showing text representation:")
```

```
print(adder.draw_circuit(output='text'))
```

```
return adder
```

Some Results

=====

QUANTUM RIPPLE CARRY ADDER DEMONSTRATION

=====

Testing: $3 + 2$

Expected result: 5

Simulation Results Analysis:

$A=6, B=2 \rightarrow \text{Sum}=5, \text{Carry}=0 \rightarrow \text{Result}=5$ (Count: 1024)

Testing: $5 + 3$

Expected result: 8

Simulation Results Analysis:

$A=5, B=6 \rightarrow \text{Sum}=0, \text{Carry}=1 \rightarrow \text{Result}=8$ (Count: 1024)

Some results

Testing: $7 + 1$

Expected result: 8

Simulation Results Analysis:

A=7, B=4 $\rightarrow$ Sum=0, Carry=1 $\rightarrow$ Result=8 (Count: 1024)

Testing: $0 + 0$

Expected result: 0

Simulation Results Analysis:

A=0, B=0 $\rightarrow$ Sum=0, Carry=0 $\rightarrow$ Result=0 (Count: 1024)

Circuit Analysis

=====

CIRCUIT ANALYSIS

=====

Total qubits used: 13

Circuit depth: 16

Number of gates: 30

Gate count breakdown:

barrier: 5

ccx: 6

cx: 9

measure: 10